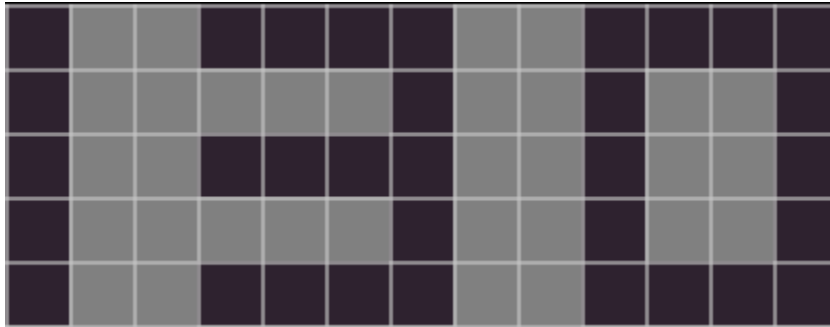


Step 1: By Hand



Step 2: Approach

We note the listed numbers of rows and columns, then we take the first list and alternate filling the listed number of spaces down each column, always starting with filled spaces, then blank spaces, alternating till the list is complete. Then we move to the next column and repeat till we are done with all lists. We need to make sure the rows are horizontal and the columns are vertical in order for our decompressed image to be readable.

Step 3: Pseudocode

```
FUNCTION decompress_image(data, num_rows, num_cols)
    // Create empty grid filled with 0s

    CREATE grid[num_rows][num_cols] initialized to 0

    // Process each column
    FOR col FROM 0 TO num_cols-1
        runs = data[col]                                // Get run-length list
    for this column
        row = 0                                          // Start at top of column

        is_filled = TRUE                                // Always start with
filled pixels

        // Process each run in the list
        FOR each length IN runs
            // Fill 'length' number of cells with current state
            FOR i FROM 1 TO length
                IF row < num_rows THEN
                    IF is_filled THEN
                        grid[row][col] = 1
                    ELSE
                        grid[row][col] = 0
```

```

        END IF
        row = row + 1
    END IF

    END FOR

    // Switch between filled and blank
    is_filled = NOT is_filled
END FOR

END FOR

RETURN grid

END FUNCTION

FUNCTION print_image(grid, filled_char='█', blank_char='░')
    FOR each row IN grid
        FOR each cell IN row
            IF cell == 1 THEN
                PRINT filled_char
            ELSE
                PRINT blank_char
            END IF
        END FOR
        PRINT new line
    END FOR
END FUNCTION

```

Step 4: Microsoft Copilot

```

def decompress_image(lists, canvas_height, canvas_width):

    # Initialize canvas with blanks (0)

    canvas = [[0 for _ in range(canvas_width)] for _ in
range(canvas_height)]

    for list_index in range(len(lists)):

        current_list = lists[list_index]

```

```
row = list_index      # start on this row
col = 0               # start at column 0
mode_fill = True      # start in FILL mode

for count in current_list:

    if mode_fill:

        # Fill horizontally for 'count' cells

        for _ in range(count):

            if col < canvas_width:

                canvas[row][col] = 1

                col += 1

    else:

        # Skip downward by 'count' rows

        row += count

        if row >= canvas_height:

            row = canvas_height - 1

        # Alternate mode

        mode_fill = not mode_fill

return canvas
```

```
def print_image(canvas, filled='■', blank='░'):
    for row in canvas:
        print(''.join(filled if cell else blank for cell in row))
```

Step 5: Compare and Contrast

Pros of My Pseudocode:

Easy to read and understand

Maintainable: Easy to edit

Cons of My Pseudocode:

Assumes the number of lists exactly matches num_cols

Less compact

Pros of Al's Pseudocode:

More compact

Creative use of alternating between “fill” and “skip” modes.

Cons of Copilot Pseudocode:

Confusing semantics

Less readable

Step 6: Update

```
FUNCTION decompress_image(runs, canvas_height, canvas_width)
    // Create a blank canvas initialized to 0 (blank)
    CREATE canvas[canvas_height][canvas_width] initialized to 0

    // Process each column vertically
    FOR col FROM 0 TO canvas_width - 1
        IF col >= length of runs THEN
```

```

        BREAK // Safety: prevent index errors if runs list is
too short

    current_runs = runs[col]

    row = 0 // Start at the top of the column

    is_filled = TRUE // Always begin with filled pixels
(per spec)

// Process each run length in this column
FOR each length IN current_runs

    // Apply this run: fill or blank 'length' cells downward
    FOR i FROM 1 TO length

        IF row < canvas_height THEN

            IF is_filled THEN

                canvas[row][col] = 1

            ELSE

                canvas[row][col] = 0

            END IF

            row = row + 1

        ELSE

            BREAK // Stop if we reach the bottom of the
canvas

        END IF

    END FOR

// Alternate between filled and blank after each run

```

```

        is_filled = NOT is_filled

    END FOR

END FOR

RETURN canvas

END FUNCTION


FUNCTION print_image(canvas, filled_char='█', blank_char='░')

    FOR each row IN canvas

        FOR each cell IN row

            IF cell == 1 THEN

                PRINT filled_char WITHOUT newline

            ELSE

                PRINT blank_char WITHOUT newline

            END IF

        END FOR

        PRINT newline

    END FOR

END FUNCTION

```